

Кафедра комп'ютерних
систем і мереж

ЛАБОРАТОРНА РОБОТА №7
з дисципліни АРХІТЕКТУРА І СТРУКТУРА ДАНИХ

Тема:

Вдосконалені алгоритми впорядкування

Виконала:
ст. гр. КІ-25-1

Лаврів А.

Перевірив:
Гарасимів Т.Г.

Мета: Набуття практичних навичок та вмінь роботи з логарифмічними алгоритмами впорядкування: пірамідальним, швидким, злиттям.

Завдання:

1. Проаналізувати приклади і доповнити частини, яких не вистачає.
2. Проілюструвати графічно роботу пірамідального сортування на прикладі впорядкування масиву з 10 цілих елементів. Масив заповнити випадковими числами від $-20N$ до $20N$ (N - номер варіанту).
3. Провести порівняльний аналіз алгоритмів. Час виконання виміряти для різної кількості елементів масиву – $100*N$, $1000*N$, $10000*N$. Масив заповнити випадковими числами від $-20N$ до $20N$ (N - номер варіанту). Дані записати в таблицю.

Код:

```
#include <iostream>
#include <cstdlib>
#include <ctime>
using namespace std;
```

```
#define N 11
#define SIZE (100*N)
```

```
void fillRandom(int a[], int size) {
    srand((unsigned)time(NULL));
    for (int i = 0; i < size; i++)
        a[i] = (rand() % (40*N + 1)) - 20*N;
}
```

```
void printArray(int a[], int size, const char* label = "") {
    if (label[0]) cout << label << ": ";
    for (int i = 0; i < size; i++)
        cout << a[i] << (i < size-1 ? ", " : "\n");
}
```

```
void copyArray(int src[], int dst[], int size) {
    for (int i = 0; i < size; i++) dst[i] = src[i];
}
```

```

void downHeap(int a[], int k, int size) {
    int child;
    int tmp = a[k];
    while (k < size / 2) {
        child = 2*k + 1;
        if ((child < size-1) && (a[child] < a[child+1]))
            child++;
        if (tmp >= a[child]) break;
        a[k] = a[child];
        k = child;
    }
    a[k] = tmp;
}

```

```

void heapSort(int a[], int size) {
    for (int i = size/2 - 1; i >= 0; i--)
        downHeap(a, i, size);

    for (int i = size - 1; i > 0; i--) {
        int last = a[i];
        a[i] = a[0];
        a[0] = last;
        downHeap(a, 0, i);
    }
}

```

```

void heapSortDemo(int a[], int size) {
    cout << "\n--- Пірамідальне сортування ---\n";
    printArray(a, size, "Вихідний масив");

    cout << "\nЕтап 1: Побудова піраміди\n";
    for (int i = size/2 - 1; i >= 0; i--) {
        downHeap(a, i, size);
        cout << " після downHeap(" << i << "): ";
        printArray(a, size);
    }
}

```

```

cout << "\nЕтап 2: Просіювання\n";
for (int i = size - 1; i > 0; i--) {
    int last = a[i];
    a[i] = a[0];
    a[0] = last;
    downHeap(a, 0, i);
    cout << "   крок " << (size - i) << ": ";
    printArray(a, size);
}

cout << "Результат: ";
printArray(a, size);
}

```

```

void quickSort(int a[], long sz) {
    long i = 0, j = sz;
    int temp, p;
    p = a[sz >> 1];

```

```

do {
    while (a[i] < p) i++;
    while (a[j] > p) j--;
    if (i <= j) {
        temp = a[i];
        a[i] = a[j];
        a[j] = temp;
        i++;
        j--;
    }
} while (i <= j);

if (j > 0) quickSort(a, j);
if (sz > i) quickSort(a + i, sz - i);
}

```

```
void quickSortWrapper(int a[], int size) {  
    if (size > 1)  
        quickSort(a, size - 1);  
}
```

```
void merge(int a[], long left, long split, long right) {  
    long pos1 = left;  
    long pos2 = split + 1;  
    long pos3 = 0;
```

```
    int* temp = new int[right - left + 1];
```

```
    while (pos1 <= split && pos2 <= right) {  
        if (a[pos1] < a[pos2]) {  
            temp[pos3] = a[pos1];  
            pos3++; pos1++;  
        } else {  
            temp[pos3] = a[pos2];  
            pos3++; pos2++;  
        }  
    }  
}
```

```
    while (pos1 <= split) {  
        temp[pos3] = a[pos1];  
        pos3++; pos1++;  
    }
```

```
    while (pos2 <= right) {  
        temp[pos3] = a[pos2];  
        pos3++; pos2++;  
    }
```

```
    for (pos3 = 0; pos3 < right - left + 1; pos3++)  
        a[left + pos3] = temp[pos3];
```

```

    delete[] temp;
}

void mergeSort(int a[], long left, long right) {
    if (left < right) {
        long split = (left + right) / 2;
        mergeSort(a, left, split);
        mergeSort(a, split + 1, right);
        merge(a, left, split, right);
    }
}

void mergeSortWrapper(int a[], int size) {
    if (size > 1)
        mergeSort(a, 0, size - 1);
}

int main() {

    const int DEMO_SIZE = 10;
    int demo[DEMO_SIZE];
    srand((unsigned)time(NULL));
    for (int i = 0; i < DEMO_SIZE; i++)
        demo[i] = (rand() % (40*N + 1)) - 20*N;

    int arr1[DEMO_SIZE];
    copyArray(demo, arr1, DEMO_SIZE);
    heapSortDemo(arr1, DEMO_SIZE);

    int arr2[DEMO_SIZE];
    copyArray(demo, arr2, DEMO_SIZE);
    cout << "\n--- Швидке сортування ---\n";
    printArray(arr2, DEMO_SIZE, "До");
    quickSortWrapper(arr2, DEMO_SIZE);
    printArray(arr2, DEMO_SIZE, "Після");
}

```

```

int arr3[DEMO_SIZE];
copyArray(demo, arr3, DEMO_SIZE);
cout << "\n--- Сортування злиттям ---\n";
printArray(arr3, DEMO_SIZE, "До");
mergeSortWrapper(arr3, DEMO_SIZE);
printArray(arr3, DEMO_SIZE, "Після");

return 0;
}

```

```

--- Пірамідальне сортування ---
Вихідний масив: -211, 59, -214, 2, 17, 58, 35, -160, 57, -71

Етап 1: Побудова піраміди
  після downHeap(4): -211, 59, -214, 2, 17, 58, 35, -160, 57, -71
  після downHeap(3): -211, 59, -214, 57, 17, 58, 35, -160, 2, -71
  після downHeap(2): -211, 59, 58, 57, 17, -214, 35, -160, 2, -71
  після downHeap(1): -211, 59, 58, 57, 17, -214, 35, -160, 2, -71
  після downHeap(0): 59, 57, 58, 2, 17, -214, 35, -160, -211, -71

Етап 2: Просіювання
  крок 1: 58, 57, 35, 2, 17, -214, -71, -160, -211, 59
  крок 2: 57, 17, 35, 2, -211, -214, -71, -160, 58, 59
  крок 3: 35, 17, -71, 2, -211, -214, -160, 57, 58, 59
  крок 4: 17, 2, -71, -160, -211, -214, 35, 57, 58, 59
  крок 5: 2, -160, -71, -214, -211, 17, 35, 57, 58, 59
  крок 6: -71, -160, -211, -214, 2, 17, 35, 57, 58, 59
  крок 7: -160, -214, -211, -71, 2, 17, 35, 57, 58, 59
  крок 8: -211, -214, -160, -71, 2, 17, 35, 57, 58, 59
  крок 9: -214, -211, -160, -71, 2, 17, 35, 57, 58, 59
Результат: -214, -211, -160, -71, 2, 17, 35, 57, 58, 59

--- Швидке сортування ---
До: -211, 59, -214, 2, 17, 58, 35, -160, 57, -71
Після: -214, -211, -160, -71, 2, 17, 35, 57, 58, 59

--- Сортування злиттям ---
До: -211, 59, -214, 2, 17, 58, 35, -160, 57, -71
Після: -214, -211, -160, -71, 2, 17, 35, 57, 58, 59

...Program finished with exit code 0
Press ENTER to exit console.

```

Рис. 1 - Демонстрація роботи коду

Алгоритм	Час (в мілісекундах)	Додаткова пам'ять	Стійкість	Природність
----------	----------------------	-------------------	-----------	-------------

	100 N	1000 N	10000N			
Пірамідальне впорядкування	169	2636	25601	Низька	Нестійкий	Низька
Швидке впорядкування	153	1670	18542	Низька	Нестійкий	Середня
Впорядкування злиттям	194	2367	23780	Висока	Стійкий	Висока

Висновок: У ході виконання лабораторної роботи було досліджено вдосконалені алгоритми впорядкування: пірамідальне сортування, швидке сортування та сортування злиттям. Було реалізовано програми для кожного алгоритму та проведено їх порівняльний аналіз.

Під час виконання роботи було встановлено, що швидке сортування є одним із найефективніших алгоритмів при роботі з великими масивами даних. Пірамідальне сортування не потребує значної додаткової пам'яті, однак є нестійким алгоритмом.